B1. Sub-RBS (Easy Version)

time limit per test 2 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1400 | Tags: combinatorics, constructive algorithms, dp, greedy, strings, two pointers |
https://codeforces.com/problemset/problem/2190/B1

This is the easy version of the problem. The difference between the versions is that in this version, you only need to evaluate for whole string *s*, *s* is *regular*, and the constraints on *n* are higher.

We say that a bracket sequence *a* is **better** than a bracket sequence *b* if one of the following holds:

- *b* is a prefix of *a*, but $a \neq b$; or
- let *i* be the first position (if it exists) where $a_i \neq b_i$, then color*red*a_i = (and color*red*b_i =).

You are given a *regular bracket sequence*□ *s* of even length *n*.

Among all non-empty **subsequences** † *t* of *s* that are *regular bracket sequences*, find the maximum possible length of *t* such that *t* is **better** than *s*. If no such *t* exists, report it.

□A regular bracket sequence is a bracket sequence that can be transformed into a correct arithmetic expression by inserting the characters 1 and + between the original characters of the sequence. For example:

- bracket sequences `()()` and `(())` are regular (the resulting expressions are `(1)+(1)` and `((1+1)+1)`);
- bracket sequences `)(`, `(`, and `)` are not.

†A sequence *a* is a subsequence of a sequence *b* if *a* can be obtained from *b* by the deletion of several (possibly, zero or all) element from arbitrary positions.

Input

Each test contains multiple test cases. The first line contains the number of test cases *t* ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer *n* ($2 \le n \le 2 \cdot 10^5$, *n* is even) — the length of the string *s*.

The second line of each test case contains a sequence *s* of length *n* consisting only of characters (and).

It is guaranteed that the given sequence *s* is a *regular bracket sequence*.

It is guaranteed that the sum of *n* over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer — the maximum possible length of a non-empty subsequence *t* of *s* that is a *regular bracket sequence* and is **better** than *s*. If no such *t* exists, print -1.

Example

Input
3 2 () 8 (()(())) 6 (())()

Output
-1 6 -1

Note

In the first example, the only non-empty regular bracket subsequence of s is $t = s =$ ( ). Since t is not better than s, we output -1.

In the second example, we can choose $t =$ ((( ))). The first index where t and s differ is $i = 3$. Since $t_3 =$ (and $s_3 =$), t is better than s. We cannot choose a longer subsequence because the only longer regular bracket subsequence is s itself, which is not better than s. Thus, we output 6.